An Analysis of Probabilistic Data Structures for Scalable, Communication-Constrained Distributed Systems

Luca Lombardo

October 26, 2025

Abstract

Scalability in distributed systems depends fundamentally on managing the tradeoff between consistency guarantees and system performance. We analyze probabilistic data structures - Bloom filters, Distributed Bloom Filters, Cuckoo filters, and HyperLogLog - as architectural solutions to two canonical problems in large-scale systems: set membership testing and cardinality estimation. Through mathematical analysis, we establish that these structures achieve information-theoretic lower bounds on space complexity while providing quantifiable probabilistic guarantees. Bloom filters require $\Theta(n \log(1/\varepsilon))$ bits to maintain a set of n elements with false positive rate ε, independent of universe size $|\mathscr{U}|$. HyperLogLog estimates cardinality with standard error $\sigma = 1.04/\sqrt{m}$ using $\Theta(m)$ registers, independent of true cardinality n. We position these structures within the PACELC consistency framework, demonstrating that they instantiate the PA/EL configuration: prioritizing Availability during partitions and Latency during normal operation, accepting probabilistic Consistency. Communication complexity analysis reveals order-of-magnitude reductions compared to exact solutions: from $\Theta(n \log|\mathscr{U}|)$ to $\Theta(n \log(1/\varepsilon))$ for set reconciliation, and from $\Theta(Nn \log|\mathscr{U}|)$ to $\Theta(Nm \log\log|\mathscr{U}|)$ for distributed cardinality aggregation. The algebraic properties of merge operations (commutativity, associativity, idempotence) enable efficient distributed aggregation without centralized coordination. We conclude that probabilistic approximation represents a principled design choice in the consistency-performance tradeoff space, essential for systems scaling to billions of operations while maintaining bounded latency.

1 Set Membership

1.1 A Space-Efficient Oracle for Set Membership

We consider the dictionary problem, which consists of maintaining a dynamic set $S \subseteq \mathscr{U}$ and supporting membership queries. In a large-scale distributed environment, classical deterministic solutions, such as hash tables, are often infeasible due to space requirements proportional to the number of elements and, more critically, the communication overhead required for synchronization [1]. The architectural constraints of such systems necessitate data structures that achieve sublinear space complexity [2]. Probabilistic data structures fulfill this requirement by trading a quantifiable measure of precision for substantial gains in space and communication efficiency. This approach enables the design of systems that prioritize performance and availability over strict consistency. We formalize this trade-off by introducing the concept of a one-sided error membership oracle.

1.1.1 Membership Queries in Large-Scale Systems

A one-sided error membership oracle guarantees correctness for negative answers, a property of paramount importance in distributed systems. A definitive negative answer allows a system to avoid expensive, high-latency operations, such as disk I/O or network round-trips, which are often the primary bottlenecks to performance [3, 2]. For instance, a storage system may consult such an oracle to determine if a specific data block might exist on a remote disk before initiating a costly read operation; a negative response obviates the need for any disk access [2, 3].

Definition 1.1 (One-Sided Error Membership Oracle)**.** A data structure $\mathscr{D}$ representing a set S is a one-sided error membership oracle with a false positive rate ε if for any element $x \in S$, the probability that $\mathscr{D}.\textsc{Query}(x)$

returns true is 1, and for any element $y \notin S$, the probability that $\mathscr{D}.\text{QUERY}(y)$ returns true is at most ε.

By definition, the oracle never produces false negatives. The adoption of such a structure is not an ad-hoc optimization but a principled architectural strategy. It represents a deliberate trade-off where a system prioritizes lower latency over strict consistency. By accepting a bounded and well-defined error probability, these algorithms enable a level of scalability and performance that would be unattainable with exact, deterministic methods [3, 2].

1.1.2 The Standard Bloom Filter

The Bloom filter is the canonical implementation of a one-sided error membership oracle [3]. It achieves its space efficiency by storing compact representations of elements, derived from hash functions, rather than the elements themselves.

Definition 1.2 (Bloom Filter [3, 4])**.** A Bloom filter for a set S with expected cardinality $|S| \leq n$ from a universe $\mathscr{U}$ consists of a bit array B of length m initialized to all zeros, and a collection of k independent hash functions $\mathscr{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathscr{U} \to \{0, 1, \ldots, m-1\}$.

To insert an element $x \in S$, we set the bits at positions $B[h_i(x)]$ to 1 for all $i \in \{1, \ldots, k\}$. To query for an element y, the operation returns true if and only if $B[h_i(y)] = 1$ for all $i \in \{1, \ldots, k\}$. A query for any element $y \in S$ is guaranteed to return true, as all corresponding bits will have been set during its insertion. This structure therefore satisfies the no-false-negatives condition of the oracle by construction.

A false positive occurs for an element $y \notin S$ if all k bits corresponding to its hash values have been incidentally set to 1 by the insertion of other elements. Assuming the hash functions distribute elements uniformly over the bit array, the probability of a specific bit remaining 0 after the insertion of n elements is $(1 - 1/m)^{kn} \approx e^{-kn/m}$. The probability of a false positive, ε, is the probability that all k bits for the queried element are 1.

Proposition 1.3 (False Positive Probability [3])**.** *The false positive probability of a Bloom filter is* $\varepsilon \approx (1 - e^{-kn/m})^k$.

A given ratio m/n of bits per element determines an optimal number of hash functions that minimizes this probability.

Theorem 1.4 (Optimal Hash Function Count [4])**.** *For a given ratio m/n, the false positive rate ε is minimized when the number of hash functions is $k = (m/n)\ln 2$.*

This theorem yields a direct relationship between the space allocated per element and the achievable false positive rate.

Corollary 1.5 (Space-Error Relationship [4])**.** *To achieve a target false positive rate ε with an optimal number of hash functions, the required number of bits per element is $m/n = -\frac{\ln \varepsilon}{(\ln 2)^2} \approx 1.44 \log_2(1/\varepsilon)$.*

The asymptotic space complexity of the Bloom filter is therefore $\Theta(n \log(1/\varepsilon))$, independent of the size of the elements. The performance relies on high-quality hash functions. In practice, computationally inexpensive non-cryptographic hash functions are sufficient, and techniques such as double hashing, which generates k hash values from two initial hashes, can reduce computational overhead without asymptotically increasing the false positive probability [3].

1.2 Limitations in Dynamic Environments

The properties of the standard Bloom filter hold under the assumption that the set S is static and its cardinality is known *a priori*. When these assumptions are violated in dynamic environments where sets change over time, two fundamental limitations emerge that restrict the filter's applicability.

First, the standard Bloom filter does not support element deletion. The INSERT operation is irreversible; clearing a bit from 1 to 0 to remove an element x would corrupt the representation of any other element y that also maps to that bit position. Such an operation would introduce false negatives, thereby violating the defining property of the one-sided error membership oracle [5, 3].

Second, the filter's capacity is fixed. Its parameters, m and k, are optimized for an expected number of elements, n. Should the actual cardinality of the set significantly exceed this estimate, the bit array becomes increasingly saturated. This saturation causes the false positive rate to degrade, approaching 1 as the number of elements grows, rendering the filter ineffective [6, 3]. These constraints necessitate architectural extensions for systems that must manage dynamic datasets of unknown or unbounded size.

1.2.1 Counting Bloom Filter

To support element deletion, the filter's structure must be augmented to track the number of elements mapped to each position. This capability was introduced as a core component of a scalable web cache sharing protocol, where the dynamic nature of cached content required a mechanism for deletions. The solution replaces the bit array with an array of counters [7].

Definition 1.6 (Counting Bloom Filter [3])**.** A Counting Bloom Filter consists of an array C of m counters, each of c bits, initialized to all zeros. It uses a collection of k independent hash functions $\mathcal{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathcal{U} \to \{0, \ldots, m-1\}$.

The operations are redefined to act upon these counters. To insert an element, we increment the counters at the k hash positions. To delete an element, we decrement the corresponding counters. A membership query returns true if and only if all corresponding counters are positive [7]. The primary trade-off is a space complexity increase by a factor of c compared to a standard Bloom filter. The selection of counter size c must be sufficient to prevent overflow; analysis has shown that 4-bit counters are sufficient for most applications [7, 3]. The arithmetic properties of the CBF are particularly useful in distributed systems for tasks such as set reconciliation, where the difference between two sets can be computed by subtracting one filter's counter array from another [8].

1.2.2 Scalable Bloom Filter

To accommodate sets of unknown or unbounded cardinality, the Scalable Bloom Filter (SBF) architecture replaces the single, fixed-size filter with a sequence of standard Bloom filters [6].

Definition 1.7 (Scalable Bloom Filter [6])**.** An SBF is an ordered sequence of s Bloom filters, $B_1, B_2, \ldots, B_s$. Each filter B_i is defined by its size m_i and number of hash functions k_i. The sequence is constructed such that for $i > 1$, $m_i > m_{i-1}$ and the target false positive rate $\varepsilon_i < \varepsilon_{i-1}$. This is typically achieved by scaling the size m_i and tightening the error rate ε_i with geometric progressions.

New elements are inserted only into the last filter in the sequence, B_s. When this filter exceeds a predefined capacity threshold (typically when its fill ratio approaches 50%), a new, larger filter B_{s+1} with a tighter error probability is appended to the sequence. A query operation must check for the element's presence in every filter, from B_s down to B_1. If any filter returns true, the query returns true. The overall false positive rate of the SBF is the probability that any of the constituent filters returns a false positive, and it can be bounded to a predefined maximum [6].

While this architecture allows the structure to grow dynamically, its application in distributed systems is severely limited by its lack of composability. The heterogeneity of the constituent filters, each with a different size m_i and number of hash functions k_i, prevents the use of meaningful bitwise operations, such as the OR operation for set union. This algebraic property is fundamental for efficient state aggregation and resource management in distributed environments, rendering the SBF unsuitable for such tasks [1].

1.3 Architectural Patterns for Distributed Systems

The deployment of Bloom filters in a distributed system, where state is partitioned or replicated across nodes, introduces challenges that transcend the data structure's inherent properties. Network latency, partitioning, and the absence of a single source of truth compel an evolution from a simple data structure to a component within a larger distributed protocol. We analyze the architectural patterns developed to manage these complexities.

1.3.1 Temporal False Negatives

Replicating a Bloom filter across multiple nodes is a common architectural pattern to reduce query latency and network load. However, maintaining instantaneous consistency across these replicas is physically impossible. Network latency ensures that a time window of state divergence exists between an update to a primary copy and its propagation to all replicas. This temporal inconsistency leads to a class of error that does not exist in single-node implementations [9].

Definition 1.8 (Temporal False Negative [9])**.** Let a global set S be represented by a primary data structure $\mathcal{D}_P$ and a set of replicas $\{\mathcal{D}_{R_1}, \ldots, \mathcal{D}_{R_n}\}$. A temporal false negative occurs at time t if an element x has been inserted into S such that a query against $\mathcal{D}_P$ would return true, but due to propagation delay, a query to a stale

replica $\mathscr{D}_{R_i}$ returns false. From the perspective of the global system state, the element is a member, yet the oracle has returned a definitive negative.

This phenomenon constitutes a violation of the core guarantee of the one-sided error membership oracle. It is not an intrinsic flaw of the Bloom filter but an emergent property of its deployment in an eventually consistent distributed system [10]. Its implications are severe, as many system designs are architected around the assumption that a 'false' response from a Bloom filter is an authoritative answer used to prevent further, more expensive operations [3, 2]. The management of this temporal inconsistency thus becomes a primary architectural concern.

1.3.2 Scalability and Consistency

The limitations of simple Bloom filter variants in distributed environments necessitate the development of comprehensive architectural patterns. These patterns address the challenges of state aggregation, parallelism, and eventual consistency not as isolated problems, but as interconnected facets of system design. We present two such synthesized architectures.

The first architecture, the Parallel Partitioned Bloom Filter (Liu2014ParBFAP), addresses the composability and performance limitations of the Scalable Bloom Filter. By constructing a sequence of homogeneous sub-filters, each sharing the same size m and number of hash functions k, it preserves the algebraic properties required for distributed operations. This homogeneity allows for the bitwise union and intersection of filters, enabling efficient state aggregation. The architecture further partitions the list of sub-filters into groups, allowing query operations to be executed in parallel across these groups, thereby improving throughput in high-concurrency systems [1].

The second and more fundamental architecture is a protocol designed to manage set reconciliation under eventual consistency. The Distributed Bloom Filter (DBF) protocol directly confronts the temporal false negative paradox by structuring interactions to guarantee convergence over time. The protocol employs unique, pair-wise hash functions for each interaction between nodes, ensuring that hash collisions are statistically independent across successive synchronization rounds.

Definition 1.9 (Distributed Bloom Filter Protocol [10])**.** Let two nodes, n_i and n_j, with unique identifiers ID_i and ID_j, wish to reconcile their local sets. The DBF protocol employs a unique, pair-wise hash function family $\mathscr{H}_{ij}$ generated from a seed s_{ij}, typically computed as $s_{ij} = ID_i \oplus ID_j$. A set of k intermediate hashes $\{h_1^{ij}, \ldots, h_k^{ij}\}$ is derived from this seed. For each element $x \in \mathscr{U}$ with a pre-computed hash $H(x)$, the final hash values are computed as:

$$H_l^{ij}(x) = (H(x) \oplus h_l^{ij}) \pmod{m} \quad \forall l \in \{1, \ldots, k\} \quad (1)$$

This method allows for the computationally inexpensive generation of unique Bloom filters for each peer interaction.

The protocol deliberately utilizes filters with a high individual false positive rate (e.g., 50%) to minimize bandwidth consumption. While a single exchange is highly probabilistic, the use of unique mappings for each interaction ensures that true set differences are revealed consistently, while random errors from false positives average out over multiple interactions. This approach sacrifices single-shot accuracy for rapid long-term convergence, a trade-off well-suited for eventually consistent systems [10]. The system-wide error rate diminishes exponentially with the number of participating nodes.

Proposition 1.10 (DBF System-Wide Error Rate [10])**.** *Let a system of N nodes use the DBF protocol, where each individual filter has a false positive rate ε. For an element $y \notin \bigcup_{i=1}^{N} S_i$, the probability that a query for y from a specific node to all of its $N-1$ peers results in a system-wide false positive is $\varepsilon_{System} \leq \varepsilon^{N-1}$.*

Proof. Let $\varepsilon = (1 - e^{-kn/m})^k$ be the FPR for a single filter. A system-wide false positive for a query from node n_j regarding an element y occurs if all $N-1$ peers it communicates with independently return a false positive. Let E_i be the event of a false positive from peer n_i. The DBF protocol's use of unique seeds s_{ji} for each peer interaction ensures that the hash function families $\mathscr{H}_{ji}$ are statistically independent for each peer i. Consequently, the events E_i are independent. The probability of a joint failure is the product of their individual probabilities: $P(\bigcap_{i=1}^{N-1} E_i) = \prod_{i=1}^{N-1} P(E_i) = \varepsilon^{N-1}$. □

This result demonstrates that even with a high individual filter error rate, the probability that all peers independently produce a false positive for the same element becomes vanishingly small as the number of

peers increases. The DBF protocol is thus an effective and bandwidth-efficient architecture for achieving eventual consistency in large-scale distributed systems [10].

1.4 Cuckoo Filters

The transition from single-node to distributed deployment exposes a structural limitation in Bloom filters that cannot be overcome through parameterization or minor refinement. In distributed systems, dataset cardinality is neither fixed nor predictable. Nodes experience churn; data undergoes compaction and replication; entries acquire expiration semantics. Elements must be deleted, not merely left to decay. A Counting Bloom Filter 1.6 addresses deletion through per-position counters, but incurs approximately four-fold space overhead compared to standard Bloom filters [3]. More fundamentally, neither variant preserves the algebraic properties required for distributed state reconciliation: the ability to compute set differences and identify true absence versus deletion.

The Cuckoo Filter presents a different architectural path. It natively supports efficient deletion in $O(1)$ time without counters or additional metadata. Furtehrmore, for false positive rates below three percent, it consumes less space than optimized Bloom filters [5]. Critically, these properties enable design patterns specifically suited to distributed scalability such as: elastic capacity growth without service interruption, co-location of filters with data partitions, and consistency models that distinguish true absence from deleted membership.

1.4.1 Partial-Key Hashing

The fundamental challenge in deploying Cuckoo Filters across network boundaries lies in the relocation operation itself. During insertion, when both candidate buckets are full, the filter must displace an existing fingerprint to its alternate location. Standard cuckoo hashing accomplishes this by recomputing the hash of the original element; the element's full key must be available to determine where to move it. In a distributed system, however, filters may be partitioned or replicated across nodes, and the original element may not be co-located with the filter metadata. Retrieving the full element for relocation decisions across network boundaries would introduce prohibitive overhead.

The solution is partial-key cuckoo hashing, introduced by Fan et al., which enables relocation decisions using only the stored fingerprint [5]. This capability is the prerequisite for all subsequent distributed patterns.

Definition 1.11 (Partial-Key Cuckoo Hashing [5])**.** For an element $x \in \mathscr{U}$ with precomputed fingerprint $f = \text{FINGERPRINT}(x)$, the two candidate bucket indices are computed as:

$$i_1 = \text{HASH}(x), \quad i_2 = i_1 \oplus \text{HASH}(f)$$

where $\oplus$ denotes bitwise XOR. There is a symmetry that allows us to, given i_2 and f, recover i_1 via the same equation: $i_1 = i_2 \oplus \text{HASH}(f)$.

This symmetry decouples relocation from full-key access. When a fingerprint must be moved from bucket i_1 to its alternate location, the system computes the destination using only f and the current position. The fingerprint can traverse network boundaries, be temporarily replicated on different nodes, or be moved as part of partition rebalancing, all without requiring access to the original element. For distributed systems where filters may be stored separately from data, or where data undergoes continuous replication and migration, this property is essential.

Definition 1.12 (Cuckoo Filter Core Operations [5])**.** A Cuckoo Filter with m buckets, each containing b entries, supports three operations:

Lookup(x): Compute $i_1 = \text{HASH}(x)$, $i_2 = i_1 \oplus \text{HASH}(f)$. Return true if fingerprint f exists in either bucket i_1 or i_2; otherwise return false. Query always examines at most $2b$ entries.

Insert(x): Compute i_1, i_2. If either bucket contains an empty entry, place f there and complete. Otherwise, select one bucket randomly, displace its resident fingerprint f', compute the alternate location for f' using $i \leftarrow i \oplus \text{HASH}(f')$, and repeat the displacement process. If relocations exceed a configurable threshold (typically 500), insertion fails and a capacity increase is required.

Delete(x): Compute i_1, i_2. If f exists in either bucket, remove one copy; otherwise return failure. Deletion requires no counters or additional bookkeeping.

We analyze the probability of a false positive, which occurs when an element not in the set hashes to a fingerprint that collides with an existing fingerprint in one of its two candidate buckets. A query inspects at most $2b$ entries across two buckets. This inspection process

yields a direct upper bound on the false positive probability, which we state formally.

Proposition 1.13 (False Positive Probability Bound [5])**.** *Let a Cuckoo Filter be configured with b entries per bucket and a fingerprint length of f bits. The false positive probability ε satisfies the inequality:*

$$\varepsilon \leq \frac{2b}{2^f}$$

From this inequality, we derive the minimum fingerprint size f required to achieve a target false positive probability ε:

$$f \geq \lceil \log_2(1/\varepsilon) + \log_2(2b) \rceil$$

For a configuration with $b = 4$ and a target $\varepsilon = 0.01$ (a 1% false positive rate), this relationship requires a fingerprint size of at least $f \geq \lceil \log_2(100) + \log_2(8) \rceil = \lceil 6.64 + 3 \rceil = 10$ bits. This yields a compact representation that is computationally sufficient to mitigate a high rate of collisions.

The architectural consequences of this design become evident when we consider the delete operation. A standard Bloom filter offers no delete mechanism; removing an element's bits risks corrupting the representation of other stored elements. A Counting Bloom Filter enables deletion but requires per-position counters, which results in a space overhead of approximately four times that of a standard Bloom filter [5]. In contrast, the Cuckoo Filter implements deletion by locating the specified fingerprint in one of its two candidate buckets and removing it. This operation requires no auxiliary data structures and directly enables the dynamic distributed architectures we analyze subsequently.

1.4.2 Capacity Scaling

In a distributed system, the cardinality of a dataset is not known *a priori*, requiring any data structure to accommodate significant growth. A Cuckoo Filter of a fixed size cannot accept new insertions once its load factor approaches its theoretical limit. While single-node deployments address this by rebuilding the filter into a larger table offline, such a procedure is untenable in distributed systems that operate under strict availability and latency constraints. A node that halts operations to rebuild its local filter violates service agreements and introduces a point of coordination that undermines distributed design principles.

The Dynamic Cuckoo Filter (DCF), introduced by Chen et al., presents a systematic solution to this scaling problem [11]. Instead of rebuilding a full filter, the DCF architecture allocates a new, empty filter and appends it to an ordered sequence of filters. All new insertions are directed to the most recently appended filter.

Definition 1.14 (Dynamic Cuckoo Filter [11])**.** A DCF is an ordered sequence of Cuckoo Filters, $\mathrm{CF}_1, \mathrm{CF}_2, \ldots, \mathrm{CF}_k$. A pointer, curCF, references the terminal filter in the sequence. An insertion operation first attempts placement on curCF. If this insertion fails due to capacity limits, a new filter is allocated, appended to the sequence, and designated as the new curCF. A query operation must traverse this sequence in reverse chronological order, from curCF to CF_1, until the target fingerprint is found or all filters have been examined.

The DCF architecture permits unbounded capacity growth without service interruption, but it achieves this at the cost of query performance. A query for an element x requires a sequential search through the filter chain. If x was inserted into an early filter and the chain has subsequently grown to a length of k, the query must inspect all k filters.

Proposition 1.15 (DCF Query Complexity)**.** *A membership query in a DCF composed of k constituent filters requires $O(k)$ bucket accesses in the worst case. For a system containing n total items, where each filter maintains a load factor $\alpha \approx 0.95$ and has a capacity of C items, the number of filters scales as $k = \Theta(n/C)$. Consequently, query latency grows proportionally with the total dataset size.*

This architectural pattern creates a direct conflict between capacity scaling and query performance. The mechanism that enables system growth is the same mechanism that causes query latency to degrade. For modern distributed systems where query latency is a critical performance metric—such as in data serving, caching, and real-time analytics—a linear growth in query time is architecturally unacceptable.

The linear query complexity arises because the DCF organizes its constituent filters temporally, by insertion order, rather than deterministically by element hash value. No direct mapping exists between an element's content and the specific filter that contains it. Membership testing therefore degenerates to a sequential search, an operation incompatible with the constant-

time query requirements of scalable distributed systems.

1.4.3 Consistent Hashing

The linear degradation in query performance of the Dynamic Cuckoo Filter arises from its temporal organization. We observe that distributed systems have already addressed analogous scaling challenges for data partitioning. Architectures such as Apache Cassandra and Amazon DynamoDB employ consistent hashing to distribute their keyspace [12, 13]. In these systems, nodes are assigned token ranges on a virtual hash ring. An element's hash value deterministically maps it to a specific point on the ring, and thus to a responsible node. The addition or removal of a node requires remapping only a localized subset of the keyspace, preserving a stable and deterministic mapping for the majority of data. Redis Cluster achieves a similar outcome using a hash slot partitioning scheme, where a fixed number of slots are distributed among nodes and can be migrated individually to scale the cluster [14].

This same principle can be applied to the organization of a Cuckoo Filter's buckets. Instead of organizing filters in a temporal sequence, we can distribute the buckets themselves across a consistent hash ring. This architectural shift aligns the filter's structure with established distributed partitioning schemes.

Definition 1.16 (Index-Independent Cuckoo Filter [15])**.** An Index-Independent Cuckoo Filter (I2CF) maintains its buckets on a consistent hash ring. For an element x with fingerprint f, its candidate bucket locations are determined by hashing the element to points on this ring. The system identifies the first bucket successor to each of these points on the ring as a candidate bucket.

The defining property of the I2CF is that bucket identifiers depend only on the element's hash value, not on the total number of buckets m in the system. Capacity expansion is achieved by adding a new bucket to a chosen position on the ring. This operation necessitates relocating only those elements whose hash values fall within the range immediately preceding the new bucket's position. For a system with n items distributed across m buckets, a rebalancing event affects an expected $O(n/m)$ items, a small fraction of the total dataset.

This design decouples capacity growth from query complexity, resolving the architectural conflict present in the DCF.

Theorem 1.17 (I2CF Query Complexity)**.** *For an I2CF with m buckets organized on a consistent hash ring, a membership query for an element x requires a sequence of constant-time operations: (1) hash computations to determine ring positions, (2) a ring lookup to identify candidate buckets (achievable in $O(1)$ expected time with an appropriate auxiliary index), and (3) inspection of entries within those buckets. The total query complexity is therefore $O(1)$, independent of the total number of items n and the total number of buckets m.*

The I2CF architecture resolves the conflation of capacity management and element mapping found in the DCF. The DCF modifies the element-to-filter mapping to accommodate growth, leading to performance degradation. In contrast, the I2CF maintains a fixed, deterministic mapping from elements to ring positions while allowing the set of buckets to expand elastically. As a result, growth and query performance become orthogonal concerns.

1.4.4 Co-Partitioning Filters with Distributed Data

We observe that the architectural value of the Index-Independent Cuckoo Filter is most fully realized when its partitioning scheme is aligned with that of the host distributed system. Many distributed key-value stores partition their keyspace across a set of nodes using a consistent hash ring. In such systems, each node assumes responsibility for one or more token ranges on the ring.

We apply this same partitioning logic to the filter's buckets, an approach whose efficacy is demonstrated by Luo et al. in their work on distributed filters over Redis Cluster [16]. We formalize this alignment as a co-partitioned filter architecture.

Definition 1.18 (Co-Partitioned Filter Architecture [16])**.** A distributed system of m nodes maintains a consistent hash ring, where each node n_i is assigned responsibility for a token range $[t_i, t_{i+1})$. The node maintains: (1) all data items d such that $H(d) \in [t_i, t_{i+1})$, and (2) all Cuckoo Filter buckets b such that $\text{position}(b) \in [t_i, t_{i+1})$. A membership query for an element x is routed to the node responsible for the token range containing $H(x)$ for local processing.

This architecture has direct performance implications. A query processed on the node that owns the element's

token range incurs only the cost of local memory access. A query originating from a different node requires a single network round-trip. The latency difference between local memory access (microseconds) and network communication (milliseconds) is typically three to four orders of magnitude.

We can model the expected query latency for this architecture under a uniform query distribution. In a system with m nodes, a randomly routed query has a probability $1/m$ of being processed locally and a probability $(1-1/m)$ of requiring network transit. This leads to the following formulation for the expected query latency.

Theorem 1.19 (Co-Partitioned Query Latency)**.** *In a co-partitioned I2CF architecture of m nodes under a uniform query distribution, the expected query latency $\mathbb{E}[L_q]$ is given by:*

$$\mathbb{E}[L_q] = \frac{1}{m} t_{local} + \left(1 - \frac{1}{m}\right) t_{network}$$

where t_{local} represents local bucket access time and $t_{network}$ represents network round-trip latency.

This latency bound is independent of the total dataset size n. Query performance is determined by hardware characteristics and system scale (m), not by the number of elements stored.

The alignment creates a synergy with the underlying infrastructure of many distributed systems. By reusing an existing consistent hashing ring, the system gains a scalable membership testing capability with minimal additional coordination overhead. Fault tolerance mechanisms, such as data replication, can be extended to include filter buckets, allowing replicated filter state to propagate through existing replication channels.

The experimental results in [16] confirm the efficacy of this approach, demonstrating significant memory savings over Bloom filter alternatives, full support for deletions, and seamless capacity scaling as nodes are added to the cluster.

1.4.5 Hierarchical Elasticity for Non-Uniform Growth Rates

While the I2CF architecture provides fine-grained, bucket-level elasticity, its rebalancing mechanism is most effective under predictable, incremental growth in dataset cardinality. Distributed systems, however, often experience non-uniform growth patterns, including sudden spikes in insertion rates. In such scenarios, where the rate of cardinality growth exceeds the rate at which new buckets can be added and populated, the system may experience cascading insertion failures.

To address this class of workloads, we analyze the Consistent Cuckoo Filter (CCF), an architecture that introduces a second, coarser tier of elasticity by organizing a collection of independent I2CF instances [15].

Definition 1.20 (Consistent Cuckoo Filter [15])**.** A CCF is a collection of I2CF instances, $\{I_1, I_2, \ldots, I_s\}$, initially with $s = 1$. Insertions are directed to a designated active instance, typically I_s. When I_s reaches a predefined load factor threshold, a new, empty instance I_{s+1} is allocated and becomes the active target for subsequent insertions. A query operation must inspect all instances in the collection until the target fingerprint is found.

This architecture allows the system to increase its total capacity instantly by adding a new, untapped I2CF. However, this "scale-out" mechanism, if left unmanaged, structurally resembles the chaining model of the Dynamic Cuckoo Filter. A query must probe every instance in the collection, leading to a performance dependency on the number of instances, s.

Proposition 1.21 (CCF Unmanaged Query Complexity)**.** *In a CCF where new, fixed-capacity I2CF instances are added sequentially without any consolidation, the number of instances s grows linearly with the total number of items n. The worst-case query complexity is therefore $O(s)$, which is equivalent to $O(n)$.*

Proof. Let n be the total number of items and let each I2CF instance have an effective capacity of C items (e.g., $C = m \cdot b \cdot \alpha$, where m is the number of buckets, b is the entries per bucket, and α is the target load factor). In an unmanaged growth model, a new instance is added every time the system has stored an additional C items. The number of instances s required to store n items is therefore $s = \lceil n/C \rceil$. As $n \to \infty$, this gives $s = \Theta(n)$. Since a query must inspect all s instances in the worst case, the query complexity is $O(s) = O(n)$. □

The CCF architecture as defined by Luo et al. explicitly includes management policies to prevent this linear degradation [15]. It is not merely a chain of filters

but a managed system that provides hierarchical scaling. This system operates at two granularities. The first is a fine-grained, bucket-level elasticity, where capacity adjustments occur within each constituent I2CF by adding or removing buckets from its consistent hash ring. The second is a coarse-grained, filter-level elasticity, which accommodates sudden load spikes by adding entire new I2CF instances. To control the growth of query latency, the architecture incorporates a consolidation mechanism, which periodically merges underutilized instances to reduce the total number of instances, s. This hierarchical approach allows the system to manage capacity at multiple granularities, handling gradual growth through the fine-grained mechanism and absorbing sudden traffic spikes by adding new instances, while maintaining long-term performance through background consolidation.

1.4.6 Concurrency Control Strategies for Distributed Cuckoo Filters

The deployment of Cuckoo Filters in a distributed environment introduces concurrency hazards. During an insertion operation, the filter performs a sequence of displacements that modify multiple bucket locations. A concurrent query examining these locations during this transient state may fail to find an element that has been evicted from its original bucket but not yet placed in its alternate location. This race condition, which we term a *false miss*, results in a definitive negative answer for an element present in the set, thereby violating the data structure's membership guarantee.

We analyze three distinct concurrency control strategies, each designed to resolve the false-miss problem for a specific system architecture and workload pattern.

The first strategy, optimistic concurrency, is designed for read-dominant workloads, such as those found in caching systems. Fan et al. developed this model for the MemC3 system [17]. The model serializes write operations while allowing multiple readers to proceed without locks, using version counters for synchronization. A key component of this model is the execution order of relocations along a displacement chain. After identifying a valid path, the system moves the last element in the chain into the empty slot first, which frees a new slot. The second-to-last element is then moved into this newly vacated slot. This reverse-order execution ensures that every element being relocated remains accessible in at least one of its two candidate buckets at all times.

Definition 1.22 (Optimistic Cuckoo Hashing [17])**.** A writer maintains a version counter for each bucket. Before initiating relocations, it identifies a valid displacement path (e.g., $a \to b \to c \to$ empty) without acquiring locks. The relocations are then executed in reverse order, with each move incrementing the version counter of the involved bucket. A reader records the versions of its two candidate buckets before inspection and rechecks them afterward. If any version has changed, the reader discards its result and retries the operation.

This strategy is highly effective for workloads where the read-to-write ratio is high, as readers do not incur lock acquisition overhead.

For general-purpose multi-core systems with more balanced read-write workloads, we analyze a fine-grained locking model that enables greater writer parallelism. The libcuckoo library[1] implements this model using lock striping, where an array of independent locks protects disjoint sets of buckets [18, 19]. A thread acquires locks only for the buckets involved in its operation, which permits concurrent write operations on non-overlapping regions of the filter.

For disaggregated memory architectures that utilize Remote Direct Memory Access (RDMA), in-memory locking mechanisms are inapplicable. Clients access remote memory directly over the network. Grant et al. developed RCuckoo for this environment, which employs a lease-based locking protocol [20]. In this model, clients acquire time-bounded leases on remote buckets via atomic RDMA operations. The automatic expiration of these leases provides fault tolerance to client failures without requiring explicit recovery protocols.

Definition 1.23 (Lease-Based Locking for RDMA [20])**.** For disaggregated memory architectures, we manage client failures during write operations through a lease-based protocol. The system maintains a separate lease table in remote memory, which is distinct from the primary lock table for normal operations. A client identifies a potentially stranded lock when an operation exceeds a predefined timeout threshold. To initiate recovery, the client acquires an exclusive repair lease for the affected index region via an atomic compare-and-swap operation on the lease table. This operation grants the client exclusive rights to modify the table state for recovery purposes. Upon completion of

[1] `https://github.com/efficient/libcuckoo`

the repair, the client relinquishes the lease by clearing its ownership flag. The repair lease itself is subject to a timeout mechanism, which provides fault tolerance against clients that fail during the recovery process itself.

We note that across all three concurrency models, the no-false-negatives property of the Cuckoo Filter remains invariant. A query that returns false provides a correct answer: the element is not, and has never been (unless subsequently deleted), a member of the set. The specific concurrency strategy is an implementation detail dictated by the hardware topology and workload, but the membership guarantee is preserved.

1.4.7 Replication and Convergence in Eventually Consistent Systems

When we replicate Cuckoo Filters across multiple nodes for fault tolerance, the replicas may temporarily diverge due to network latency in propagating updates. We analyze the consequences of this divergence, particularly for deletion operations, and compare the behavior of Cuckoo Filters to that of standard and Counting Bloom Filters.

A standard Bloom filter provides no mechanism for deletion; elements, once inserted, remain in the filter indefinitely. A Counting Bloom Filter supports deletion by decrementing counters but at the cost of an approximately four-fold increase in space overhead [5]. A Cuckoo Filter, by contrast, implements deletion as a direct removal of the element's fingerprint.

In a replicated environment, this distinction becomes architecturally significant. Consider a Cuckoo Filter replicated on nodes n_1 and n_2. An insertion of element x on n_1 propagates asynchronously to n_2. During the propagation window, a query for x on n_2 will return false. This behavior, a form of temporal false negative, is inherent to any asynchronously replicated system.

The key difference emerges in deletion scenarios, a problem central to systems that use tombstones to mark deleted data under eventual consistency [21]. When element x is deleted on n_1, its fingerprint is immediately removed from the local filter. During the propagation window, a query for x to n_1 correctly returns false, while a query to n_2 returns a stale positive. Once the deletion operation propagates to n_2, its fingerprint is also removed, and both nodes converge to a consistent state where x is correctly identified as absent.

A Bloom filter behaves differently. Since it cannot remove the element, it will continue to return positive for the deleted key indefinitely. This forces the system to perform expensive validation lookups against the backing data store merely to discover a tombstone, negating much of the filter's performance benefit in workloads with frequent deletions [21].

Proposition 1.24 (Cuckoo Filter Replication Convergence)**.** *In an eventually consistent system with replicated Cuckoo Filters, let an element x be inserted at time t_i and deleted at time $t_d > t_i$. For any replica r, there exist propagation delays such that the insertion is applied at time $\tau_i \geq t_i$ and the deletion is applied at time $\tau_d \geq t_d$. For all times $t > \tau_d$, a query for x on replica r will deterministically return false. In contrast, a standard Bloom filter provides no upper bound on the time until a query for a deleted element ceases to return a positive result.*

This property enables Cuckoo Filters to converge more rapidly to the true state of the system, especially in environments with frequent deletions. For distributed systems implementing eventual consistency, this allows the filter's state to accurately track operations such as the purging of tombstones during compaction or garbage collection. The ability to remove fingerprints, rather than merely marking them for future cleanup, ensures that the filter remains a reliable oracle for the live, non-deleted state of the dataset.

1.4.8 Impact on Distributed System Architectures

We analyze the concrete architectural impact of Cuckoo Filters in several domains where distributed systems require both efficient membership testing and dynamic element deletion.

Our first application domain is Log-Structured Merge-Tree (LSM-tree) based key-value stores. In these systems, data is organized into immutable, sorted files on disk (SSTables), each typically indexed by an in-memory filter. Deletions are often handled by writing a tombstone marker rather than by immediate data removal [21]. A standard Bloom filter, being unable to remove entries, continues to return a positive match for a deleted key. This behavior forces the database to perform a disk I/O operation only to discover the tombstone, incurring unnecessary latency. A Cuckoo Filter resolves this inefficiency. During data compaction,

the fingerprint corresponding to a purged tombstone can be removed from the filter. Subsequent queries for the deleted key will correctly return false, eliminating the wasted I/O. The Chucky architecture further exploits this by unifying the multiple per-level filters of an LSM-tree into a single Cuckoo Filter, which reduces the point query complexity from O(number of levels) to $O(1)$ [22].

In named-data networking (NDN), we examine the Pending Interest Table (PIT), which tracks outstanding content requests. The DiCuPIT system utilizes distributed Cuckoo Filters to replace traditional Bloom filter-based implementations. This architectural change yields a 36% reduction in lookup time and a 31% reduction in memory consumption compared to Bloom filter baselines [23]. The primary advantage is the ability to correctly and efficiently delete PIT entries as their corresponding data responses arrive, a fundamental operation that standard Bloom filters cannot support.

Our third domain is state synchronization in blockchain networks. Nodes in these systems maintain a mempool of pending transactions. To propagate a new block efficiently, the Gauze protocol employs Cuckoo Filters to summarize the contents of a node's mempool [24]. A node propagating a block sends this compact filter to its peers. The receiving nodes use the filter to identify which transactions in the new block are absent from their local mempool and request only this delta. This method reduces block propagation bandwidth by 40 to 50%. The efficiency gain is a direct result of the filter's compact representation; a filter for a million-element set requires only a few megabytes of space.

These applications demonstrate a shared architectural requirement: efficient support for both insertion and deletion in resource-constrained environments, where the primary bottlenecks are network bandwidth, I/O latency, or processing throughput. The ability to delete elements distinguishes the Cuckoo Filter from the Bloom filter not as an incremental improvement, but as an enabling architectural property. It facilitates correct state reconciliation, efficient rebalancing under dynamic scaling, and accurate membership semantics in systems where the lifecycle of data includes explicit deletion.

2 Cardinality Estimation

Having established the architectural necessity of probabilistic oracles for set membership queries, we now address the related but quantitatively distinct challenge of estimating the number of unique elements within a large-scale dataset. This task, formally known as the cardinality estimation problem, is a fundamental primitive in data profiling, network monitoring, and large-scale analytics [25].

Definition 2.1 (The Cardinality Estimation Problem)**.** Given a multiset S of elements drawn from a universe $\mathcal{U}$, the cardinality estimation problem is to determine $|S|$, defined as the number of distinct elements in S.

In a distributed, data-parallel framework, a naive implementation of this task necessitates a multi-stage execution. In an initial stage, each parallel worker processes a partition of the input multiset to produce a local set of unique elements. Subsequently, the framework must perform a shuffle operation, transmitting all unique elements from all partitions across the network to a common set of reducer nodes for final aggregation and deduplication [26].

This shuffle operation constitutes the primary architectural bottleneck. The communication cost of this approach is not fixed; it is proportional to the number of unique elements and their size. For datasets with high cardinality, the network I/O required for the shuffle dominates the total execution time, rendering the approach impractical for interactive or large-scale analytics. This communication overhead represents a fundamental physical constraint that makes exact, deterministic solutions for distributed cardinality estimation architecturally unsustainable [26].

2.1 The HyperLogLog Algorithm

The HyperLogLog (HLL) algorithm provides a composable, fixed-size data structure, or sketch, that offers a probabilistic solution to the cardinality estimation problem. By constructing a compact summary of a dataset, it replaces the network-intensive shuffle of raw data with the transmission of the sketch itself, thereby transforming a communication-bound problem into one that is computationally manageable. Its design is predicated on the principles of stochastic averaging and the use of a harmonic mean for robust aggregation.

2.1.1 Stochastic Averaging and the Harmonic Mean

We sum up the core mechanics of the HyperLogLog algorithm as introduced by Flajolet et al. [27]. The algorithm maps elements to an array of registers and records an observable based on the bit patterns of their hash values.

Definition 2.2 (HyperLogLog Sketch)**.** A HyperLogLog sketch consists of an array M of $m = 2^p$ registers, where p is a precision parameter. The sketch is associated with a hash function $h : \mathcal{U} \to \{0,1\}^L$ that maps elements from a universe $\mathcal{U}$ to binary strings of sufficient length L. Each register $M[j]$ is initialized to 0.

The algorithm processes a multiset by updating this array of registers. For each element, a single hash value is used to select a register and compute an update value. This process, known as stochastic averaging, uses one hash function to simulate m independent experiments, with each register representing one experiment.

To insert an element $v \in S$, we first compute its hash value $x = h(v)$. The first p bits of x are used to determine a register index $j \in \{0,\dots,m-1\}$. Let w be the remaining $L-p$ bits of x. We then compute an observable $\rho(w)$, defined as one plus the number of leading zeros in w. The corresponding register is updated according to the rule $M[j] := \max(M[j], \rho(w))$. After all elements have been processed, each register $M[j]$ contains the maximum ρ value observed for any element mapped to that register.

The final cardinality estimate is derived from the state of these registers. The algorithm uses the harmonic mean of the values $2^{M[j]}$, which is less sensitive to outlier values than the arithmetic or geometric mean.

Proposition 2.3 (HyperLogLog Estimator [27])**.** *Given a HyperLogLog sketch with m registers M, the cardinality estimate E is computed as:*

$$E = \alpha_m m^2 \left(\sum_{j=0}^{m-1} 2^{-M[j]} \right)^{-1}$$

where α_m is a bias correction constant that depends on m.

This mechanism produces a fixed-size sketch of $\Theta(m \log\log n)$ bits that summarizes the cardinality of a set containing n elements. The theoretical relative error of this estimate is approximately $1.04/\sqrt{m}$ [27]. The critical property for distributed systems is that these fixed-size sketches can be efficiently combined.

2.2 The HyperLogLog Algorithm

The HyperLogLog (HLL) algorithm provides a composable, fixed-size data structure, or sketch, that offers a probabilistic solution to the cardinality estimation problem. By constructing a compact summary of a dataset, it replaces the network-intensive shuffle of raw data with the transmission of the sketch itself, thereby transforming a communication-bound problem into one that is computationally manageable. Its design is predicated on the principles of stochastic averaging and the use of a harmonic mean for robust aggregation.

2.2.1 Stochastic Averaging and the Harmonic Mean

We formalize the core mechanics of the HyperLogLog algorithm as introduced by Flajolet et al. [27]. The algorithm maps elements to an array of registers and records an observable based on the bit patterns of their hash values.

Definition 2.4 (HyperLogLog Sketch)**.** A HyperLogLog sketch consists of an array M of $m = 2^p$ registers, where p is a precision parameter. The sketch is associated with a hash function $h : \mathcal{U} \to \{0,1\}^L$ that maps elements from a universe $\mathcal{U}$ to binary strings of sufficient length L. Each register $M[j]$ is initialized to 0.

The algorithm processes a multiset by updating this array of registers. For each element, a single hash value is used to select a register and compute an update value. This process, known as stochastic averaging, uses one hash function to simulate m independent experiments, with each register representing one experiment.

To insert an element $v \in S$, we first compute its hash value $x = h(v)$. The first p bits of x are used to determine a register index $j \in \{0,\dots,m-1\}$. Let w be the remaining $L-p$ bits of x. We then compute an observable $\rho(w)$, defined as one plus the number of leading zeros in w. The corresponding register is updated according to the rule $M[j] := \max(M[j], \rho(w))$. After all elements have been processed, each register $M[j]$ contains the maximum ρ value observed for any element mapped to that register.

The final cardinality estimate is derived from the state of these registers. The algorithm uses the harmonic mean of the values $2^{M[j]}$, which is less sensitive to outlier values than the arithmetic or geometric mean.

Proposition 2.5 (HyperLogLog Estimator [27])**.** *Given a HyperLogLog sketch with m registers M, the cardinality estimate E is computed as:*

$$E = \alpha_m m^2 \left(\sum_{j=0}^{m-1} 2^{-M[j]} \right)^{-1}$$

where α_m is a bias correction constant that depends on m.

This mechanism produces a fixed-size sketch of $\Theta(m \log\log n)$ bits that summarizes the cardinality of a set containing n elements. The theoretical relative error of this estimate is approximately $1.04/\sqrt{m}$ [27]. The critical property for distributed systems is that these fixed-size sketches can be efficiently combined.

2.2.2 Union

The utility of the HyperLogLog sketch in distributed systems is derived from its support for a computationally efficient union, or merge, operation. This operation allows sketches computed on disparate data partitions to be combined to produce a sketch representing the union of the underlying sets.

Definition 2.6 (HyperLogLog Union [28])**.** Let M_1 and M_2 be two HyperLogLog sketches of m registers, representing sets S_1 and S_2 respectively. The union sketch, $M_{1\cup 2}$, representing the set $S_1 \cup S_2$, is computed as the register-wise maximum of the two input sketches:

$$M_{1\cup 2}[j] := \max(M_1[j], M_2[j]) \quad \forall\, j \in \{0, \ldots, m-1\}$$

The resulting sketch $M_{1\cup 2}$ is identical to the sketch that would have been generated by processing all elements from both S_1 and S_2 in a single pass.

The register-wise `max` operation endows the HLL union with a set of algebraic properties that are fundamental to its scalability in parallel and distributed computations.

Theorem 2.7 (Algebraic Properties of the Union Operation [28])**.** *The HyperLogLog union operation is commutative, associative, and idempotent. For any sketches M_A, M_B, and M_C:*

- *Commutativity:* $M_A \cup M_B = M_B \cup M_A$
- *Associativity:* $(M_A \cup M_B) \cup M_C = M_A \cup (M_B \cup M_C)$
- *Idempotency:* $M_A \cup M_A = M_A$

These properties have direct architectural implications. Commutativity and associativity guarantee that the final result of merging a collection of sketches is independent of the order of aggregation. This allows a system to perform reductions in parallel, in a hierarchical tree structure, or in any arbitrary sequence without requiring coordination or ordering constraints. Idempotency ensures that re-merging a duplicate sketch does not alter the aggregate state. This property simplifies fault-tolerance protocols, as it allows for message delivery with at-least-once semantics, obviating the need for more complex exactly-once delivery mechanisms for sketch aggregation [28].

Collectively, these properties enable the non-additive `COUNT(DISTINCT)` problem to be reframed as a decomposable aggregation problem, analogous to a `SUM`. This allows the computation to be parallelized with minimal communication overhead, a defining characteristic for scalable distributed systems [26].

2.3 Architectural Implications

While the algebraic properties of the HyperLogLog algorithm provide a theoretical foundation for its use in distributed systems, deployment at scale exposes practical challenges that are not apparent from the initial formulation. When applied to common analytical workloads, such as large-scale group-by aggregations, the standard HLL algorithm encounters limitations related to estimation accuracy at low cardinalities and communication overhead. These challenges motivated a series of engineered refinements, culminating in the HyperLogLog++ algorithm [28].

2.3.1 Challenges at Scale

We analyze two primary challenges that arise when deploying the standard HyperLogLog algorithm in a data-parallel execution environment. Both are consequences of the data distribution patterns common in large-scale analytical queries.

First, analytical workloads frequently involve a high degree of cardinality skew. In a distributed `GROUP BY` operation executed over billions of records, the resulting groups often follow a power-law distribution. A small number of groups may have very high cardinalities, but the vast majority of groups will contain very few unique elements [28]. The original HLL estimator exhibits significant systematic bias and a larger relative error for

these low-cardinality sets, which compromises the accuracy of the estimates for the long tail of the distribution.

Second, the aggregation of a massive number of sketches introduces a new form of communication overhead. In a distributed `GROUP BY` query, the map phase generates a separate HLL sketch for each group on each worker node. While each individual sketch is small, the total number of sketches can be in the millions or billions. The subsequent shuffle phase must serialize, transmit, and deserialize this large volume of small objects. This process can itself become a network and CPU bottleneck, undermining the efficiency gains achieved by avoiding the shuffle of raw data [28].

2.3.2 HyperLogLog++

The HyperLogLog++ algorithm introduces a series of engineering refinements to the standard HLL to address the challenges of accuracy and communication overhead at scale [28]. These modifications are not merely optimizations but architectural enablers for large-scale distributed analytical systems.

First, to address the systematic bias at low cardinalities, HyperLogLog++ replaces the small-range correction with a bias correction mechanism derived from empirical data. The system uses a pre-computed table of bias values obtained from extensive simulations to correct the raw estimate, significantly improving accuracy for the low-cardinality groups prevalent in skewed analytical workloads [28].

Second, it mandates the use of a 64-bit hash function. This change expands the hash space to make collisions statistically insignificant even for cardinalities exceeding billions, thereby eliminating the need for the large-range correction formula of the original algorithm and simplifying the implementation [28].

The most significant architectural refinement is the introduction of a dual-representation sketch that adapts its storage format based on cardinality. This design directly addresses the communication overhead associated with shuffling a massive number of sketches.

Definition 2.8 (Sparse and Dense Representations [28])**.** A HyperLogLog++ sketch can exist in one of two formats:

- A `dense` format, which is the standard HLL representation: a fixed-size array of m registers.
- A `sparse` format, used for low cardinalities, which stores only the non-zero registers as a compressed, sorted list of (index, value) pairs.

The sketch begins in the `sparse` format and transitions to the `dense` format only when its size in the sparse representation would exceed the size of the dense format.

The use of a sparse representation is a fundamental enabler for interactive distributed analytics. In group-by queries with high cardinality skew, the majority of generated sketches represent very few elements. The sparse format reduces the memory footprint of these numerous low-cardinality sketches by an order of magnitude or more. This reduction in size directly translates to a commensurate reduction in the volume of data that must be serialized, transmitted during the shuffle phase, and deserialized by reducer nodes. This transformation of the communication cost makes HLL++ a viable architectural component for systems that require low-latency responses to large-scale aggregation queries [28].

2.4 Architectures and Implementations

The algebraic properties of HyperLogLog sketches, particularly their efficient and composable union operation, enable their integration into diverse distributed computing architectures. The choice of architecture is typically dictated by the nature of the data workload, whether batch or streaming, and the desired query patterns.

2.4.1 Architectural Patterns

We analyze three canonical architectural patterns for distributed HLL computation. The first is the MapReduce paradigm, fundamental for large-scale batch processing. In this model, an input dataset is partitioned across worker nodes. The map stage generates local HLL sketches for each data partition, creating a separate sketch for each grouping key in a group-by operation. The system then shuffles these compact sketches to reducer nodes. In the reduce stage, each reducer applies the union operation to merge the sketches it receives for a given key. The correctness of this stage, where sketches may arrive in an arbitrary order, is formally guaranteed by the commutativity and associativity of the union operation [26]. This pattern drastically

reduces network I/O compared to an exact distributed distinct count.

The second pattern is designed for real-time aggregation in stream processing systems. This architecture partitions an unbounded data stream into logical windows, such as fixed time intervals or element counts. For each active window, an operator maintains a local HLL sketch. The composability of sketches allows for efficient computation over combined windows. To compute the cardinality over a time interval T composed of sub-intervals $t_1, \ldots, t_k$, the system retrieves the corresponding sketches $M_{t_1}, \ldots, M_{t_k}$ and computes the final sketch as $M_T = \bigcup_{i=1}^{k} M_{t_i}$. The cost of this operation is independent of the raw data volume within the interval, enabling efficient, stateful stream processing [29].

The third pattern is a hierarchical, or tree-based, aggregation model, well-suited for systems with a natural data hierarchy [30]. In this model, leaf nodes generate sketches from raw data streams. Intermediate nodes in the hierarchy periodically collect and merge the sketches from their children to create higher-level aggregates. The validity of this hierarchical model rests upon the associativity of the HLL union, which ensures that the final global sketch is identical regardless of the tree's depth or the order of intermediate merge operations. This pattern enables multi-resolution querying, where queries for cardinality at different levels of the hierarchy can be answered by accessing pre-aggregated sketches without reprocessing data from lower levels.

2.4.2 Implementation Case Studies

The architectural patterns enabled by HyperLogLog are realized in a spectrum of production systems, ranging from low-level data structures requiring explicit user orchestration to high-level, transparent query optimizations managed entirely by a database engine. We analyze three representative implementations.

The Redis implementation provides HLL as a primitive data type, offering maximum flexibility. It exposes three core commands: `PFADD` to insert elements, `PFCOUNT` to retrieve the cardinality estimate, and `PFMERGE` to perform the union of multiple sketches. An HLL sketch is represented as a fixed-size string. This design choice facilitates a client-driven model for distributed computation: an application can retrieve a serialized sketch from one node, transmit it over the network, and merge it with another sketch on a different node by invoking `PFMERGE`. In this model, the orchestration of the distributed aggregation is the explicit responsibility of the application logic.

In contrast, distributed SQL query engines such as Presto and Google BigQuery integrate HLL as a high-level, transparent optimization within the query execution layer. These systems expose the functionality through an aggregate function, typically `APPROX_DISTINCT`. When executing a distributed query, the query planner automatically pushes the sketch creation stage to the worker nodes. The workers compute HLL++ sketches, often utilizing both sparse and dense representations to optimize for memory and network usage. In the final stage, these partial sketches are shuffled to a reducer or coordinator node, which performs a final merge aggregation. The entire process is managed by the query optimizer, abstracting the underlying mechanics from the user [28]. BigQuery further provides a multi-stage API (`HLL_COUNT.INIT`, `HLL_COUNT.MERGE`) that allows intermediate, partially aggregated sketches to be materialized in tables, enabling efficient incremental rollups [31].

The highest level of abstraction is represented by systems like the Citus extension for PostgreSQL. This system provides transparent distributed cardinality estimation through automatic query rewriting. An administrator can enable approximate counting via a configuration parameter. Subsequently, the distributed query planner automatically rewrites standard `COUNT(DISTINCT)` queries to use an HLL-based implementation. It pushes sketch aggregation down to the worker nodes and merges the intermediate sketches on the coordinator node to produce the final result. From the user's perspective, the use of HLL is a transparent physical-layer optimization managed entirely by the database [25].

2.5 Advanced Challenges in Production Environments

The deployment of probabilistic data structures in production systems introduces challenges beyond simple performance metrics. We analyze three such challenges: ensuring fault tolerance efficiently, providing security against adversarial manipulation, and managing privacy in federated contexts.

2.5.1 Fault Tolerance and Approximate Recovery

In data-parallel frameworks, fault tolerance for stateless transformations is achieved by recomputing lost partitions from their lineage graph [26]. For stateful operations, such as maintaining HLL sketches over streaming data, this approach is insufficient. Traditional fault tolerance for stateful operators requires periodic checkpointing of the entire state, which incurs significant I/O overhead and compromises performance [29].

An alternative approach, termed approximate fault tolerance, leverages the inherently probabilistic nature of the data structure to create more efficient recovery mechanisms. Instead of checkpointing at fixed intervals, this model triggers a backup only when the potential error introduced by a failure would exceed a user-defined threshold. This is managed by monitoring the divergence between the current in-memory state and its last persisted backup [29].

Definition 2.9 (State Divergence [29])**.** Let $M_{current}$ be the current state of an HLL sketch and M_{backup} be its most recent persisted state. The state divergence, $D(M_{current}, M_{backup})$, is a metric quantifying the difference between the two sketches. The AF-Stream framework defines this divergence based on the specific properties of the streaming algorithm being used. For HLL, this can be modeled as a function of the differences between the register values.

The AF-Stream protocol executes a backup operation only when the state divergence D exceeds a preconfigured threshold Θ, or when the number of unprocessed items since the last backup exceeds another threshold L. This adaptive strategy significantly reduces backup overhead during normal operation. The framework provides a formal guarantee that the total accumulated error after any number of failures remains bounded [29].

Proposition 2.10 (Bounded Error Under Failures [29])**.** *In the AF-Stream model, let a worker experience k failures. The total divergence between the actual state and the ideal, failure-free state is bounded by a function of the user-specified thresholds Θ and L, and algorithm-specific constants. The error bound is independent of the number of failures k.*

This model demonstrates a principled trade-off, where a system accepts a small, provably bounded increase in estimation error in the rare event of a failure in exchange for a significant reduction in the constant overhead of checkpointing.

2.5.2 Security Against Adversarial Manipulation

The probabilistic nature of HyperLogLog, while providing efficiency, also introduces security vulnerabilities. We analyze a specific class of adversarial action, the cardinality inflation attack, where a malicious actor seeks to deliberately produce an arbitrarily large cardinality estimate from a small set of crafted inputs [32].

The attack leverages the internal structure of the HLL algorithm. An attacker can empirically discover input elements that produce high ρ values for each of the m registers. Even without knowledge of the specific hash function, an attacker can construct a malicious set by repeatedly generating random inputs and observing their effect on a local HLL instance. By collecting just m such inputs, one targeting each register, an attacker can construct a set that populates every register with a high value. When this small set is inserted into the target HLL sketch, the harmonic mean calculation yields a massive, artificially inflated cardinality estimate. This vulnerability has been demonstrated to be effective against production implementations in systems such as Presto and Redis [32].

We consider two classes of mitigation strategies. The first involves cryptographic techniques, such as applying a secret, system-specific salt to the hash function. This approach makes it computationally difficult for an external attacker to craft malicious inputs offline. However, this strategy breaks the mergeability property for sketches that use different salts, limiting its applicability in systems that rely on sketch union for aggregation across different administrative domains [32].

The second class of strategies involves robust aggregation mechanisms designed to operate in the presence of Byzantine adversaries. Instead of a simple `max` operation during a merge, or a simple harmonic mean for estimation, the system can employ aggregation rules that are resilient to outliers. Techniques from the field of Byzantine-resilient distributed optimization, such as the Geometric Median or aggregation rules like Krum, are designed to filter out malicious updates [33]. The Geometric Median, for instance, finds a point that minimizes the sum of Euclidean distances to a set of input vectors, making it inherently robust to arbitrarily corrupted inputs. Applying such robust aggregators to the

vector of HLL registers during a merge operation is an active area of research for building secure, Byzantine-fault-tolerant analytical systems [34].

2.5.3 Privacy Implications in Federated Systems

While HLL sketches provide a compact summary that does not store raw data elements, their use in federated environments introduces privacy considerations. The act of sharing a sketch, even if it contains only anonymized hash-derived values, can still leak information about the underlying dataset, particularly concerning unique or rare elements [33].

In a federated query system, an adversary who compromises the central aggregator or observes the transmitted sketches could perform linkage attacks. If a hospital finds that only one patient satisfies the criteria for a query, sharing an HLL sketch of this single-element set still reveals information. An attacker could potentially infer properties of this unique patient by correlating the sketch with a known background population [33]. To quantify this risk, we can apply the concept of k-anonymity to the buckets of an HLL sketch.

Definition 2.11 (Bucket k-Anonymity [33])**.** Let B be the set of patients at a hospital matching a query and let A be the background population of all patients at that hospital. An HLL bucket is defined to be k-anonymous if the value it contains could have been generated by at least k distinct patients from the background population A. An attacker who observes the sketch cannot determine with certainty which of the k individuals contributed to that bucket's value.

The level of k-anonymity provided by an HLL sketch is a function of the sketch parameters, the size of the background population, and the size of the query set. Research has shown that it is possible to compute theoretical bounds on the expected number of buckets in a sketch that are not k-anonymous. HLL sketches are particularly effective for rare diseases (where the ratio of query set size to background population size is low). This is in marked contrast to sharing exact counts, where a count of one immediately breaks anonymity [33]. This analysis allows system designers to manage the trade-off between the accuracy of the cardinality estimate, which increases with the number of buckets m, and the privacy risk, as a larger m may decrease the expected k-anonymity of each bucket.

2.6 Comparative Analysis and Future Directions

We conclude our analysis by situating the HyperLogLog algorithm within the broader landscape of probabilistic data structures for set summarization. This comparison highlights the specific architectural trade-offs that make HLL a suitable choice for certain classes of problems, and delineates areas of ongoing research.

2.6.1 Functional Trade-offs and Algorithmic Specialization

The architectural value of a probabilistic data structure is determined not only by its space-error characteristics but also by the set of algebraic operations it supports. The HyperLogLog algorithm is highly specialized for cardinality estimation and set union. While its merge operation is fundamental for distributed aggregation, it does not natively support other set-theoretic operations such as intersection or set difference.

Alternative data structures, such as MinHash and its derivatives like Theta Sketches, provide a richer set of supported operations. These sketches are designed to estimate the Jaccard similarity between sets, from which the sizes of their union, intersection, and difference can be derived. However, this increased functionality comes at a cost. For the primary task of cardinality estimation, HLL is substantially more space-efficient than MinHash-based sketches providing the same estimation accuracy [35].

This distinction represents a fundamental trade-off between functional generality and architectural efficiency. HLL is architecturally optimized for the specific problem of distributed, union-based cardinality aggregation. Its design provides a near-optimal solution for this problem class. For analytical queries that require measuring the overlap between sets, a different class of data structures is necessary. The choice between these algorithms is therefore not an optimization detail but a primary architectural decision dictated by the functional requirements of the system.

2.6.2 The Frontier of Cardinality Estimation

Research in cardinality estimation continues to refine the space-error trade-off and explore new architectural possibilities. Recent algorithmic developments demonstrate that further space efficiencies are achiev-

able while preserving the essential algebraic properties required for distributed deployment.

The UltraLogLog algorithm modifies the underlying register structure to encode more information per bit [36]. It generalizes the HyperLogLog register by partitioning its bits. A set of q bits stores the maximum update value u_i seen so far, while a set of d additional bits acts as a compact summary, indicating whether updates with values $u_i - 1, \ldots, u_i - d$ have occurred. This design, particularly for the ULL configuration with $q = 6$ and $d = 2$, fits into a single 8-bit register. It achieves a theoretical memory-variance product approximately 28% lower than a standard 6-bit HLL sketch while remaining commutative, idempotent, and mergeable.

The HyperLogLogLog algorithm achieves compression through a sparse-dense architecture based on value offsets [37].

Definition 2.12 (HyperLogLogLog Sketch Structure [37])**.** A HyperLogLogLog sketch is a 3-tuple (S, M, B) where:

- B is a base value, representing a lower bound for the majority of register values.
- M is a dense array storing small-width integer offsets for registers whose values are close to B. For a register j where $B \leq M'[j] < B + 2^k$, its offset $M[j] = M'[j] - B$ is stored using k bits.
- S is a sparse associative array storing pairs $(j, M'[j])$ for outlier registers whose values fall outside the dense offset range.

The base value B is chosen dynamically to minimize the total representation size $k \cdot m + |S| \cdot (\log m + \log w)$.

This structure adapts to the statistical distribution of register values, using compact offsets for the common case and a sparse map for exceptions. The resulting sketch requires approximately $m \log_2 \log_2 \log_2 m$ bits for large n, albeit with a potential increase in the computational complexity of updates [37].

Several open problems remain at the intersection of cardinality estimation and distributed systems. The development of robust aggregation protocols that are resilient to Byzantine adversaries is a critical area of research. Securing sketch-based systems requires moving beyond simple merge logic to robust aggregation rules that can tolerate corrupted inputs. Furthermore, integrating formal privacy guarantees with cardinality estimation sketches is necessary for enabling collaborative analytics on sensitive data. Finally, new architectural paradigms such as sketch disaggregation, where a single logical sketch is fragmented and distributed across network hardware, present a frontier for research into ultra-low-latency, in-network analytics [38].

References

[1] Yi Liu, Xiongzi Ge, D. Du, and Xiaoxia Huang. Parbf: A parallel partitioned bloom filter for dynamic data sets. *The International Journal of High Performance Computing Applications*, 30:259 – 275, 2014.

[2] Fay W. Chang, J. Dean, S. Ghemawat, Wilson C. Hsieh, D. Wallach, M. Burrows, Tushar Chandra, A. Fikes, and R. Gruber. Bigtable: a distributed storage system for structured data. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 205–218, 2006.

[3] Sasu Tarkoma, Christian Esteve Rothenberg, and Eemil Lagerspetz. Theory and practice of bloom filters for distributed systems. *IEEE Communications Surveys & Tutorials*, 14:131–155, 2012.

[4] A. Broder and M. Mitzenmacher. Network applications of bloom filters: A survey. *Internet Mathematics*, 1:485 – 509, 2004.

[5] Bin Fan, D. Andersen, M. Kaminsky, and M. Mitzenmacher. Cuckoo filter: Practically better than bloom. *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies*, 2014.

[6] Paulo Sérgio Almeida, Carlos Baquero, Nuno M. Preguiça, and D. Hutchison. Scalable bloom filters. *Inf. Process. Lett.*, 101:255–261, 2007.

[7] Li Fan, P. Cao, J. Almeida, and A. Broder. Summary cache: a scalable wide-area web cache sharing protocol. In *Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, volume 28, pages 254–265, 1998.

[8] Deke Guo and Mo Li. Set reconciliation via counting bloom filters. *IEEE Transactions on Knowledge and Data Engineering*, 25:2367–2380, 2013.

[9] Yifeng Zhu and Hong Jiang. False rate analysis of bloom filter replicas in distributed systems. *2006 International Conference on Parallel Processing (ICPP'06)*, pages 255–262, 2006.

[10] Lum Ramabaja and Arber Avdullahu. The distributed bloom filter. *ArXiv*, abs/1910.07782, 2019.

[11] Hanhua Chen, Liangyi Liao, Hai Jin, and Jie Wu. The dynamic cuckoo filter. *2017 IEEE 25th International Conference on Network Protocols (ICNP)*, pages 1–10, 2017.

[12] A. Lakshman and Prashant Malik. Cassandra: a decentralized structured storage system. *ACM SIGOPS Oper. Syst. Rev.*, 44:35–40, 2010.

[13] Giuseppe DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: amazon's highly available key-value store. In *Symposium on Operating Systems Principles*, pages 205–220, 2007.

[14] Redis. Redis Cluster Specification. `https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/`, 2025. Accessed: 2025-10-26.

[15] Lailong Luo, Deke Guo, Ori Rottenstreich, Richard T. B. Ma, Xueshan Luo, and Bangbang Ren. A capacity-elastic cuckoo filter design for dynamic set representation. *IEEE Transactions on Network and Service Management*, 18:4860–4874, 2021.

[16] Baozhou Luo, Wenjun Zhu, Peng Li, and Zhijie Han. Distributed dynamic cuckoo filter system based on redis cluster. *2018 IEEE 4th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing, (HPSC) and IEEE International Conference on Intelligent Data and Security (IDS)*, pages 244–247, 2018.

[17] Bin Fan, D. Andersen, and M. Kaminsky. Memc3: Compact and concurrent memcache with dumber caching and smarter hashing. In *Symposium on Networked Systems Design and Implementation*, pages 371–384, 2013.

[18] Xiaozhou Li, D. Andersen, M. Kaminsky, and M. Freedman. Algorithmic improvements for fast concurrent cuckoo hashing. In *European Conference on Computer Systems*, pages 27:1–27:14, 2014.

[19] Wenhai Li, Zhiling Cheng, Yuan Chen, Ao Li, and Lingfeng Deng. Lock-free bucketized cuckoo hashing. In *European Conference on Parallel Processing*, pages 275–288, 2023.

[20] Stewart Grant and A. Snoeren. Cuckoo for clients: Disaggregated cuckoo hashing. In *USENIX Annual Technical Conference*, pages 959–972, 2025.

[21] Sharafat Ibn Mollah Mosharraf and M. A. Adnan. Improving lookup and query execution performance in distributed big data systems using cuckoo filter. *Journal of Big Data*, 9, 2021.

[22] Niv Dayan and M. Twitto. Chucky: A succinct cuckoo filter for lsm-tree. In *SIGMOD Conference*, 2021.

[23] Arman Mahmoudi and M. Ahmadi. Dicupit: Distributed cuckoo filter-based pending interest table. *ArXiv*, abs/2308.02801, 2023.

[24] Xiaoqiang Ding, Liushun Zhao, Lailong Luo, Junjie Xie, Deke Guo, and Jinxi Li. Gauze: enabling communication-friendly block synchronization with cuckoo filter. *Frontiers of Computer Science*, 17(3):173403, 2023.

[25] Hazar Harmouch and Felix Naumann. Cardinality estimation: An experimental survey. *Proc. VLDB Endow.*, 11:499–512, 2017.

[26] M. Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, M. Franklin, S. Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Symposium on Networked Systems Design and Implementation*, pages 15–28, 2012.

[27] P. Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. *Discrete Mathematics & Theoretical Computer Science*, pages 137–156, 2007.

[28] Stefan Heule, Marc Nunkesser, and Alexander Hall. Hyperloglog in practice: algorithmic engineering of a state of the art cardinality estimation algorithm. In *International Conference on Extending Database Technology*, pages 683–692, 2013.

[29] Qun Huang and P. Lee. Toward high-performance distributed stream processing via approximate fault tolerance. *Proc. VLDB Endow.*, 10:73–84, 2016.

[30] I. Wilms and J. Bien. Tree-based node aggregation in sparse graphical models. *Journal of machine learning research : JMLR*, 23, 2021.

[31] Ziye Tao, G. Weber, and Y. Yu. Expected 10-anonymity of hyperloglog sketches for federated queries of clinical data repositories. *Bioinformatics*, 37:i151 – i160, 2021.

[32] P. Reviriego, Pablo Adell, and Daniel Ting. Hyperloglog (hll) security: Inflating cardinality estimates. *ArXiv*, abs/2011.10355, 2020.

[33] Tuan Le and Shana Moothedath. Byzantine resilient federated multi-task representation learning. *ArXiv*, abs/2503.19209, 2025.

[34] K. Kuwarancharoen and S. Sundaram. On the geometric convergence of byzantine-resilient distributed optimization algorithms. *SIAM J. Optim.*, 35:210–239, 2023.

[35] Otmar Ertl. Setsketch: Filling the gap between minhash and hyperloglog. *ArXiv*, abs/2101.00314, 2021.

[36] Otmar Ertl. Ultraloglog: A practical and more space-efficient alternative to hyperloglog for approximate distinct counting. *Proc. VLDB Endow.*, 17:1655–1668, 2023.

[37] Matti Karppa and R. Pagh. Hyperlogloglog: Cardinality estimation with one log more. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[38] Jonatan Langlet, Peiqing Chen, Michael Mitzenmacher, Ran Ben Basat, Zaoxing Liu, and Gianni Antichi. Sketch disaggregation across time and space. *ArXiv*, abs/2503.13515, 2025.

[39] Zhinan Cheng, Qun Huang, and P. Lee. On the performance and convergence of distributed stream processing via approximate fault tolerance. *The VLDB Journal*, 28:821 – 846, 2018.